Extending a Single Cycle MIPS Processor with JAL, JR, MULT, MFHI, MFLO, MIN, MAX and SUM

CSE332 Computer Architecture and Organization, Assignment 5, Summer 2026

Group Member 1
Name Mustakim Ahmed Hasan
Student ID 2421349042

Group Member 2
Name Md. Rafiul Haque
Student ID 2413697042

Group Member 3
Name Md Ashikur Rahman
Student ID 2411632642

Group Member 4
Name Sami Hyder Chowdhury
Student ID 2132720642

Abstract. *This report describes the work we did to extend a single cycle MIPS processor written in Verilog. The design we were given already handled the shift instructions SLL and SRL, but it had no support for jump and link or for jump register, and it had no multiply unit at all. We added JAL, JR, MULT, MFHI and MFLO, and then three custom instructions of our own called MIN, MAX and SUM. Most of the work sits in the control unit, although each new instruction also needs a varying amount of extra hardware in the datapath, ranging from none at all for MIN and MAX up to a whole multiplier and a new pair of registers for MULT. We give the full control signal truth table, two annotated figures showing what we added, worked encoding examples taken from a real assembler, and the results we measured after running test programs on the finished processor. Every value the processor produced matched what we had worked out by hand.*

Index Terms. *MIPS, single cycle processor, Verilog, control unit, datapath, computer architecture.*

I. INTRODUCTION

We were handed two folders. One held a single cycle MIPS processor written in Verilog and the other held a MIPS assembler written in C++. The processor folder was named WOJAL, which is short for without JAL, and that name turned out to be an accurate description of what was missing.

The design could already fetch, decode and execute the common arithmetic, logic, memory and branch instructions. It could also perform SLL and SRL, because the ALU cases and the shift amount wiring were both in place from the start. What it could not do was call a function and return from one, and it had no way to multiply two numbers.

Our job was to add JAL and JR. After finishing those we also added the multiply family, which is made up of MULT, MFHI and MFLO. We did all of the work on Windows and never needed WSL. Section II explains what each new instruction has to do, Section III describes the hardware we added, and Section IV gives the truth table. The remaining sections cover the encodings, the control unit code, and the testing we carried out.

II. WHAT THE NEW INSTRUCTIONS DO

JAL is a J type instruction. It jumps to a target address in the same way that J does, and at the same time it saves the address of the following instruction, which is PC plus 4, into register 31. Register 31 is also known as $ra. That saved address is what allows a function to return to whoever called it.

JR is an R type instruction with a function field of 001000. It reads a register, normally $ra, and copies that value straight into the program counter. It writes no register of its own.

MULT is also R type. It multiplies the two source registers as signed numbers and produces a 64 bit result. That result is too large for a general purpose register, so MIPS keeps it in two dedicated registers called HI and LO. HI holds the upper 32 bits and LO holds the lower 32 bits.

MFHI and MFLO are the instructions that read those two dedicated registers back out again. MFHI copies HI into rd and MFLO copies LO into rd.

MIN, MAX and SUM are not part of the MIPS instruction set at all. They are custom instructions we added on top. MIN writes the smaller of its two source registers into rd and MAX writes the larger, both treating the values as signed. SUM writes the sum of the two source registers into rd. It is worth being upfront that SUM performs exactly the same operation as ADD. It exists as a separately decoded instruction with its own function code rather than as a genuinely new operation.

III. WHAT WE ADDED TO THE DATAPATH

A. Jump and link

JAL has to do two separate things inside the same clock cycle. Taking the jump was the easy half, because the jump target path already existed and we only had to raise the Jump signal that was already there. Saving the return address was the half that needed new hardware, because nothing in the original datapath could force a particular destination register or feed PC plus 4 into the register file.

We solved it by widening two muxes that were already present. The mux that picks the write register used to be a two way choice between rt and rd. We turned it into a four way mux so that JAL can force the constant 31. The mux that picks the write back value used to be a two way choice between the ALU result and the memory read data, and we widened that one as well so that JAL can select PC plus 4. As it happens the starter code already contained an unused four way mux module called mux4, so we did not have to write one from scratch.

B. Jump register

JR needed a genuinely new path rather than a wider version of an old one. Every way of producing the next program counter in the original design came either from an adder or from bits of the instruction itself. None of them could take a value out of the

register file. We added one more two way mux after the existing jump mux. While JR is low that mux simply passes on whatever the earlier muxes chose, and when JR is high it passes the value sitting on read data 1, which is the contents of $ra.

C. Multiply, move from HI and move from LO

The multiply family needed the most new hardware. We added a signed 64 bit multiplier driven by the two register file outputs, together with a new pair of clocked registers called HI and LO which live in a new file named hilo.v. The control signal MultOp acts as the write enable for that pair, so the product is only stored when a MULT instruction is actually executing.

Reading the values back needed one more mux on the write back path. When MFHiLo is high, the value written into rd comes from HI or LO instead of from the usual write back mux. Choosing between HI and LO turned out to need no new control signal at all. MIPS numbers MFHI as 010000 and MFLO as 010010, so bit 1 of the function field already tells them apart, and we used that bit directly as the select line.

One detail is worth stating clearly. MULT and JR both have to hold RegWrite low, because neither of them writes a general purpose register. MFHI and MFLO are the opposite case. They are ordinary R type writes to rd, so they keep the normal R type values for RegDst and RegWrite.

D. Minimum, maximum and sum

These three needed no new control signals whatsoever, which makes them the cheapest additions in the whole project. They are plain three operand R type instructions that read two registers, do something in the ALU, and write the result to rd. That is exactly what the existing R type defaults already describe, so RegDst, RegWrite, ALUSrc and MemtoReg all keep the values they already had. The only thing the control unit does is select the right ALU operation.

The real obstacle was somewhere else entirely. The ALU operation is selected by a field called ALUControl, and that field was four bits wide. Fifteen of its sixteen possible codes were already spoken for, which left exactly one free code, and we needed three. So we widened ALUControl from four bits to five. That change touches four files, because the field runs from the control unit through the datapath and into the ALU.

Widening a field that everything depends on sounds risky, but it was made safe by giving every existing operation its old code with a leading zero added. ADD was 0000 and became 00000, SLT was 1000 and became 01000, and so on for all fifteen. Nothing that previously worked changed meaning. The three new codes then live in the upper half of the widened space, which was entirely empty before. We re ran all three earlier test programs afterwards and every result was identical to what it had been, which is what we would expect if the widening was done correctly.

For the function codes we picked 101100, 101101 and 101110. These are unused in the real MIPS R type encoding space, so the new instructions cannot be confused with any genuine MIPS instruction.

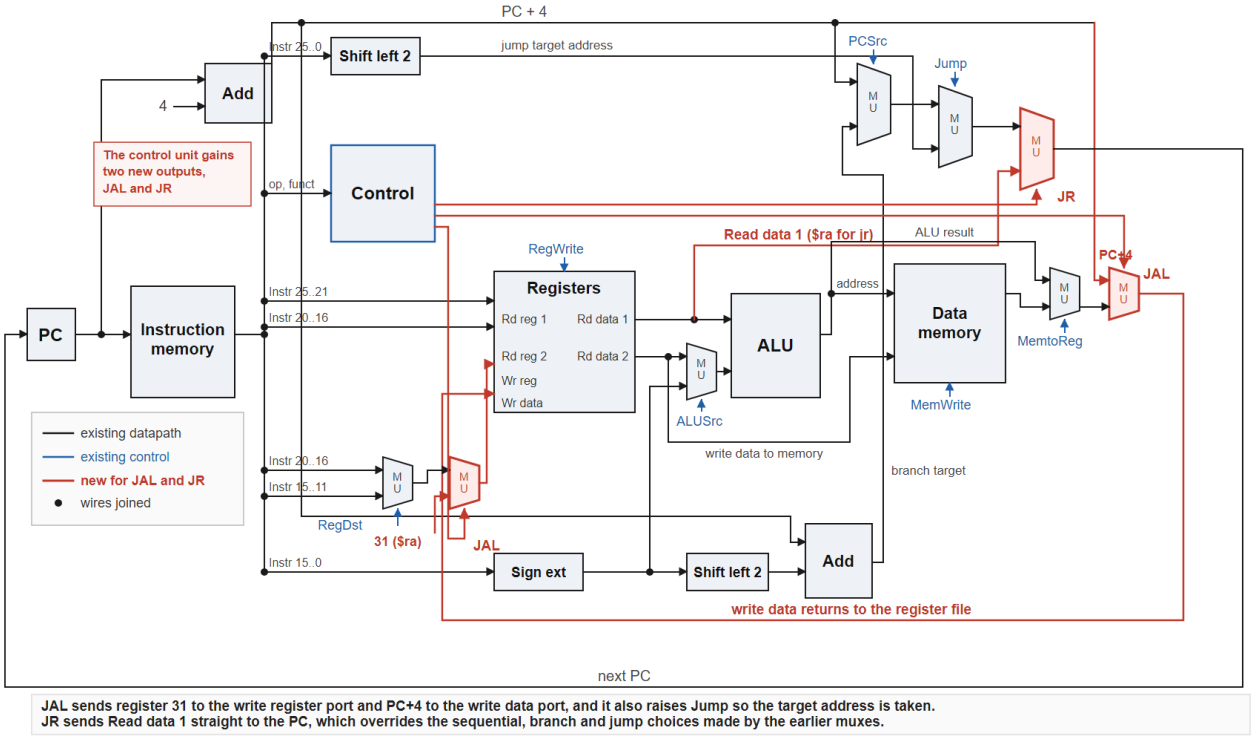

Fig. 1. The single cycle datapath with the JAL and JR additions drawn in red.

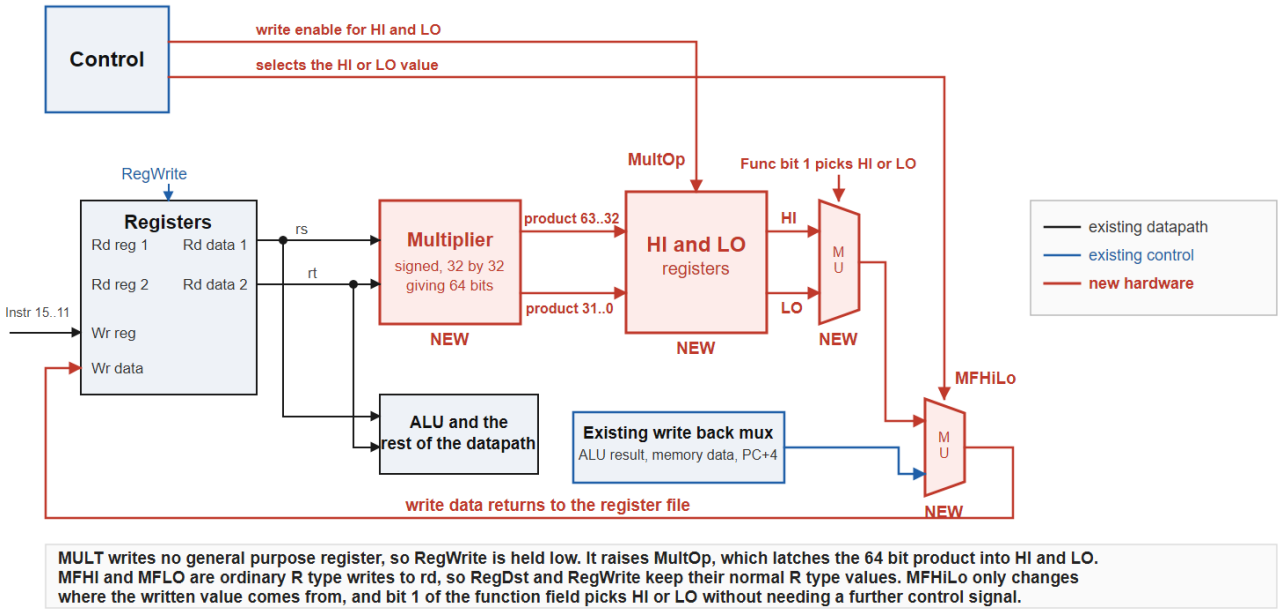

Fig. 2. The hardware added for MULT, MFHI and MFLO. Parts of the datapath that do not change are shown as single blocks.

TABLE I
CONTROL SIGNAL VALUES FOR EVERY SUPPORTED INSTRUCTION

Instruction	RegDst	ALUSrc	MemToReg	RegWrite	MemWrite	Branch, BNE	Jump	JAL	JR	MultOp	MFHiLo	ALUcont
R type ALU ops	1	0	0	1	0	0, 0	0	0	0	0	0	per op
sll	1	0	0	1	0	0, 0	0	0	0	0	0	00101
srl	1	0	0	1	0	0, 0	0	0	0	0	0	00110
sra	1	0	0	1	0	0, 0	0	0	0	0	0	00111
sllv	1	0	0	1	0	0, 0	0	0	0	0	0	01011
srlv	1	0	0	1	0	0, 0	0	0	0	0	0	01100
srav	1	0	0	1	0	0, 0	0	0	0	0	0	01101
jr (added)	X	X	X	0	0	0, 0	0	0	1	0	0	X
mult (added)	X	0	X	0	0	0, 0	0	0	0	1	0	X
mfhi (added)	1	0	X	1	0	0, 0	0	0	0	0	1	X
mflo (added)	1	0	X	1	0	0, 0	0	0	0	0	1	X
min (added)	1	0	0	1	0	0, 0	0	0	0	0	0	10000
max (added)	1	0	0	1	0	0, 0	0	0	0	0	0	10001
sum (added)	1	0	0	1	0	0, 0	0	0	0	0	0	10010
lw	0	1	1	1	0	0, 0	0	0	0	0	0	00000
sw	X	1	X	0	1	0, 0	0	0	0	0	0	00000
beq	X	0	X	0	0	1, 0	0	0	0	0	0	00001
bne	X	0	X	0	0	1, 1	0	0	0	0	0	00001
addi	0	1	0	1	0	0, 0	0	0	0	0	0	00000
addiu	0	1	0	1	0	0, 0	0	0	0	0	0	00000
andi	0	1	0	1	0	0, 0	0	0	0	0	0	00010
ori	0	1	0	1	0	0, 0	0	0	0	0	0	00011
xori	0	1	0	1	0	0, 0	0	0	0	0	0	00100
slti	0	1	0	1	0	0, 0	0	0	0	0	0	01000
sltiu	0	1	0	1	0	0, 0	0	0	0	0	0	01001
j	X	X	X	0	0	0, 0	1	0	0	0	0	X
jal (added)	X	X	X	1	0	0, 0	1	1	0	0	0	X
lui	0	1	0	1	0	0, 0	0	0	0	0	0	01110

The five shaded rows are the instructions we added. An X marks a signal whose value does not matter for that instruction. The control unit combines the Branch and BNE bits into PCSrc using the expression PCSrc equals Branch AND the result of Zero XOR BNE.

TABLE II
ENCODINGS PRODUCED BY THE ASSEMBLER FOR THE ADDED INSTRUCTIONS

Instruction	Machine code	Field breakdown
`jal func`	`0x0C000007`	opcode 000011, address field 7, which is byte address 0x1C divided by 4
`jr $ra`	`0x03E00008`	opcode 000000, rs 11111 which is $ra, function field 001000
`sll $t2, $a0, 2`	`0x00045080`	opcode 000000, rt 00100, rd 01010, shift amount 00010, function 000000
`srl $t3, $t2, 1`	`0x000A5842`	opcode 000000, rt 01010, rd 01011, shift amount 00001, function 000010
`mult $t0, $t1`	`0x01090018`	opcode 000000, rs 01000, rt 01001, function field 011000
`mflo $t2`	`0x00005012`	opcode 000000, rd 01010, function field 010010
`mfhi $t3`	`0x00005810`	opcode 000000, rd 01011, function field 010000
`min $s0, $t0, $t1`	`0x0109802C`	opcode 000000, rs 01000, rt 01001, rd 10000, function field 101100
`max $s1, $t0, $t1`	`0x0109882D`	opcode 000000, rs 01000, rt 01001, rd 10001, function field 101101
`sum $s2, $t0, $t1`	`0x0109902E`	opcode 000000, rs 01000, rt 01001, rd 10010, function field 101110

We checked every encoding above against the MIPS instruction formats by hand. The jump target field is worth a note, because the assembler divides the byte address by four before placing it in the instruction, which is why a jal to address 0x1C carries the value 7.

IV. THE MODIFIED CONTROL UNIT

The control unit gained four new outputs. JAL and JR handle the two jump instructions, while MultOp and MFHiLo handle the multiply family. All four default to zero at the top of the always block so that no latch is inferred for the instructions that do not use them.

```
output reg JAL,
output reg JR,
output reg MultOp,
output reg MFHiLo,
...
always @(*) begin
  JAL    <= 1'b0;
  JR     <= 1'b0;
  MultOp <= 1'b0;
  MFHiLo <= 1'b0;
```

Inside the R type branch we added four new function codes. JR and MULT raise their own signal, while MFHI and MFLO share MFHiLo because the function field bit already separates them.

```
6'b001000: begin              // JR
    ALUControl <= 5'b00000;
    JR <= 1'b1;
  end
6'b011000: begin              // MULT
    ALUControl <= 5'b00000;
    MultOp <= 1'b1;
  end
6'b010000: begin              // MFHI
    ALUControl <= 5'b00000;
    MFHiLo <= 1'b1;
  end
6'b010010: begin              // MFLO
    ALUControl <= 5'b00000;
    MFHiLo <= 1'b1;
  end
```

MIN, MAX and SUM sit in the same R type branch but are far simpler, because selecting an ALU operation is the only thing they need.

```
6'b101100: ALUControl <= 5'b10000;  // MIN
6'b101101: ALUControl <= 5'b10001;  // MAX
6'b101110: ALUControl <= 5'b10010;  // SUM
```

The matching ALU entries, where a and b are the two source register values, read as follows. Both comparisons are wrapped in a signed cast, which is what makes negative operands behave correctly.

```
5'b10000: y = $signed(a) < $signed(b) ? a : b;
5'b10001: y = $signed(a) > $signed(b) ? a : b;
5'b10010: y = a + b;
```

JAL is decoded from its own opcode rather than from a function field, because it is a J type instruction.

```
6'b000011: begin              // JAL
    temp <= 8'b10000010;
    ALUControl <= 5'b00000;
    JAL <= 1'b1;
  end
```

Finally, JR and MULT have to cancel the RegWrite bit that the R type default switched on. We do that after the packed temp vector has been unpacked, and we test the opcode and function inputs directly rather than testing our own JR and MultOp outputs. The reason for that choice is explained in Section VI.

```
{RegWrite,RegDst,ALUSrc,Branch,
 MemWrite,MemtoReg,Jump,BNE} = temp;

if (Opcode == 6'b000000 &&
    Func == 6'b001000) RegWrite = 1'b0;
if (Opcode == 6'b000000 &&
    Func == 6'b011000) RegWrite = 1'b0;
```

V. TESTING AND RESULTS

The assignment did not ask for a testbench, but we wanted to know whether the design actually worked rather than only looking correct on paper, so we tested it anyway.

First we rebuilt the assembler we were given. The compiled binary that shipped with it turned out to be an ARM64 Linux executable, which cannot run on Windows under any circumstances. That is almost certainly why WSL was suggested to us. Rebuilding the C++ source with the MSYS2 compiler produced a normal Windows executable and removed the need for WSL entirely.

We then wrote two short assembly programs, assembled them, converted the output into the hex format that the instruction memory expects, and ran them in Icarus Verilog. Icarus is a free Verilog simulator that runs natively on Windows, and it uses exactly the same source files that ModelSim will use later.

The first program calls a function with jal, returns from it with jr, and then performs a shift left and a shift right. The processor finished with $a0 equal to 5, $v0 equal to 105, $ra equal to 12, $t1 equal to 105, $t2 equal to 20 and $t3 equal to 10. The value in $t1 is the important one, because it could only have been produced if jr returned control to the correct instruction.

The second program multiplies negative five by five and then reads both halves of the answer back. We deliberately chose a negative operand so that the test would catch a multiplier that got the low word right but the sign extension wrong. HI came out as 0xFFFFFFFF and LO came out as 0xFFFFFFE7, which together represent minus 25 as a 64 bit signed value, and mflo and mfhi copied those into $t2 and $t3 correctly.

After adding the multiply hardware we reran both earlier programs to confirm that nothing had broken, and the results were identical to before.

The third program tests MIN, MAX and SUM, and it also writes its answers into data memory so that the results survive somewhere we can inspect after the run has finished. It computes min and max

and sum of 25 and 7, and then repeats min and max using 25 against negative 12.

That negative operand is deliberate. A test built only from positive numbers would pass whether the comparison was signed or unsigned, so it would prove very little. Negative 12 read as an unsigned 32 bit number is roughly 4.29 billion, which is larger than 25 rather than smaller. So an unsigned comparison would report the minimum of 25 and negative 12 as 25, and the maximum of negative 12 and 7 as negative 12. Our processor returned negative 12 and 7 respectively, which is only possible with a correctly signed comparison.

The register results were min of 25 and 7 gives 7, max of 25 and 7 gives 25, sum of 25 and 7 gives 32, min of 25 and negative 12 gives negative 12, and max of negative 12 and 7 gives 7. All five match hand calculation.

The five values were then stored to consecutive words of data memory, and the testbench dumped the whole of data memory to a file at the end of the run. The relevant part of that dump is shown below. Every word outside the first five is zero, which is itself a useful check, because it confirms the stores landed exactly where they were addressed and did not disturb anything else.

```
// 0x00000000
00000007     min(25, 7)   = 7
00000019     max(25, 7)   = 25
00000020     sum(25, 7)   = 32
fffffff4     min(25, -12) = -12
00000007     max(-12, 7)  = 7
00000000
00000000
00000000
...           all remaining words are zero
```

The fourth entry is worth a second look. Negative 12 stored as a 32 bit two's complement value is fffffff4 in hexadecimal, which is what appears in memory. This confirms the negative result was not only computed correctly but also written to memory intact.

VI. PROBLEMS WE RAN INTO

Three faults in the starter files had to be fixed before any simulation would run at all. None of them are related to the instructions we added, but all of them blocked progress.

The control unit computed PCSrc from a signal called B which was never declared anywhere in the file. The intended signal was clearly BNE. The instruction memory tried to read a file called memory.txt, but the only memory image supplied with the project is called memfile.txt. The testbench had no finish statement, so it would run forever in a non interactive simulator.

The most interesting problem appeared while we were testing JR. Our first attempt wrote to the packed temp vector from two nested case scopes inside one combinational always block, once for the R type default and once again for the JR override. That leaves two pending non blocking assignments to the same variable in a single pass, and the simulator scheduler never settles. The simulation did not crash or print an error. It simply sat at time zero forever.

The fix was to stop writing temp twice. We assign it once, then apply the JR and MULT overrides afterwards using the opcode and function inputs, which are stable. It is a useful thing to know about, because a simulation that hangs with no error message gives you very little to go on.

VII. CONCLUSION

We added eight instructions to the single cycle MIPS design we were given. JAL and JR were the two required by the assignment, MULT, MFHI and MFLO followed afterwards, and MIN, MAX and SUM were added last as custom instructions of our own. The amount of hardware each one needed varied a great deal. MIN, MAX and SUM needed no new control signals at all and only a wider ALU control field. JAL needed wider versions of muxes that already existed. JR needed a new path into the program counter. The multiply family needed a multiplier and a new pair of registers.

We verified all of them in simulation against values we had calculated by hand, and we did the entire project natively on Windows. The same Verilog files will run unchanged in ModelSim when it becomes available to us.